What we've learnt so far

- Mov instruction and various forms of memory operand
  - `movl 16(%rdi, %rsi, 4), %eax`
- Arithmetic instructions
- Lea instruction

Today's lesson plan

How is non-linear control flow realized?

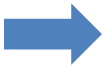
- RFLAGS register and how arithmetic instructions set it
- Special instructions that also set RFLAGS
  - **`cmp, test`**
- Instructions that read RFLAGS to set register values
  - **`set`**
- Instructions that (read RFLAGs to) set %rip
  - **`jmp`**

How is non-linear control flow realized?

```
if .. else

for (i=0; i<10; i++) {..}

while (str[i] != ‘\0’) {..}
```



???

Control flow uses RFLAGS register

PC: Program counter
- Store memory address of next instruction
- Also called “RIP” in x86_64

IR: instruction register
- Store the fetched instruction

General purpose registers:
- Store operands and pointers used by program

Program status and control register:
- Contain status of the instruction executed
- Called “**RFLAGS**” in x86_64

How control flow uses RFLAGS register

- RFLAGS is a special purpose register
- Different bits represent different status flags
  - All useful bits reside in the lower 32 bits (EFLAGS)
- Certain instructions set status flags
  - Regular arithmetic instructions: **`add, sub`**
  - Special flag-setting instructions: **`cmp, test, set`**
- **jmp** instructions use flags to set %rip

EFLAGS register: ZF

- ZF (Zero Flag):
  - Set if the result of the instruction is zero; cleared otherwise.

```
movq $2, %rax
subq $2, %rax
```

EFLAGS register: SF

- SF (Sign Flag):
  - Set to be the most-significant bit of the result.

```
movq $2, %rax
subq $10, %rax
```

EFLAGS register: CF

- CF (Carry Flag):
  - Set if an unsigned arithmetic operation carries (or borrows) out of msb.

  ➡ i.e. Set if overflow for unsigned integer arithmetic

```
movq $0xffffffffffffffff, %rax
addq $2, %rax
```

```
movq $0, %rax
subq $1, %rax
```

EFLAGS register: OF

- OF (Overflow Flag):
  - Set if there is carry-in but no carry-out of msb
  - or, there is no carry-in but there's carry-out of msb



Set if overflow for signed integer (2's complement) arithmetic.

```
movq $0x7fffffffffffffff, %rax
addq $1, %rax
```

```
movq $0x8000000000000000, %rax
addq $0xffffffffffffffff, %rax
```

CF and OF are different flags

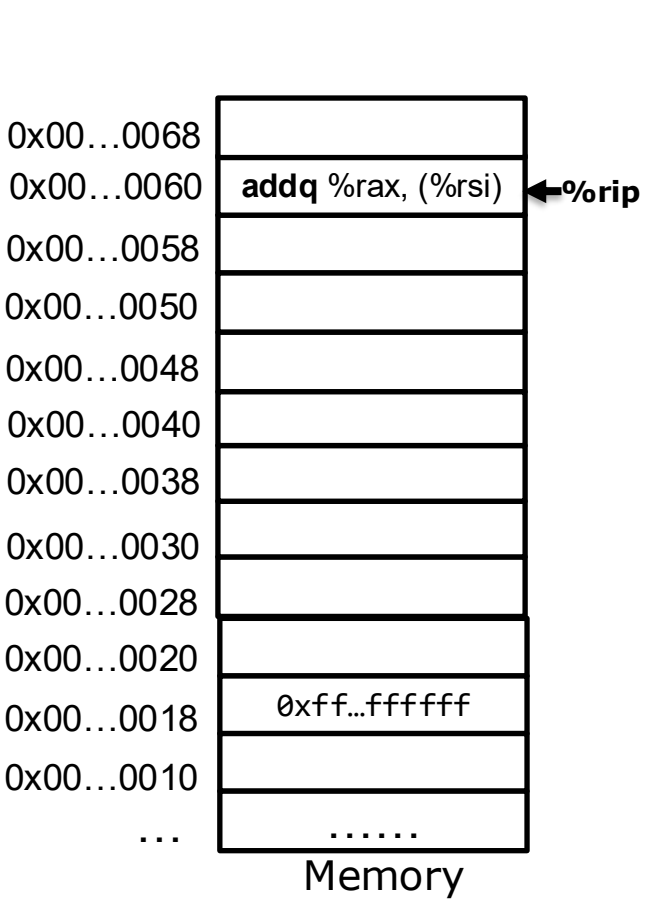
- CPU does not care if data represents signed or unsigned integer:
  - Same underlying arithmetic hardware circuitry.
  - Both OF and CF flags are set by examining various carry bits
- Up to programmer/compiler to interpret either CF or OF flag

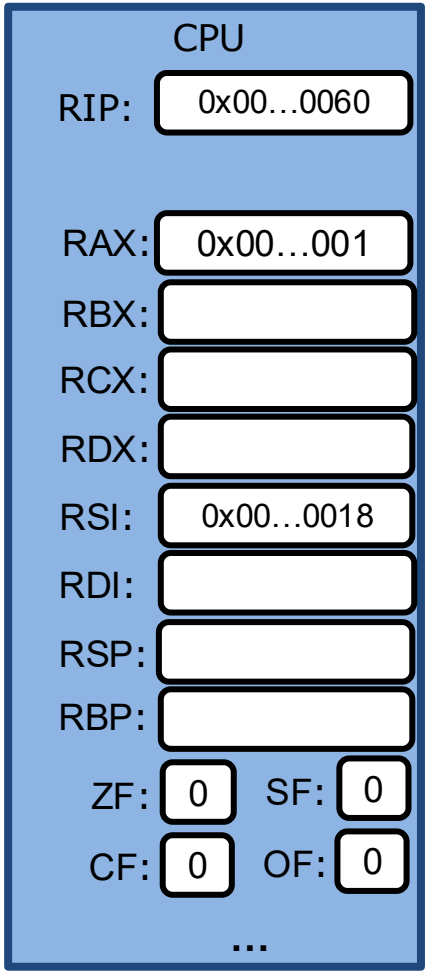
Status flags summary

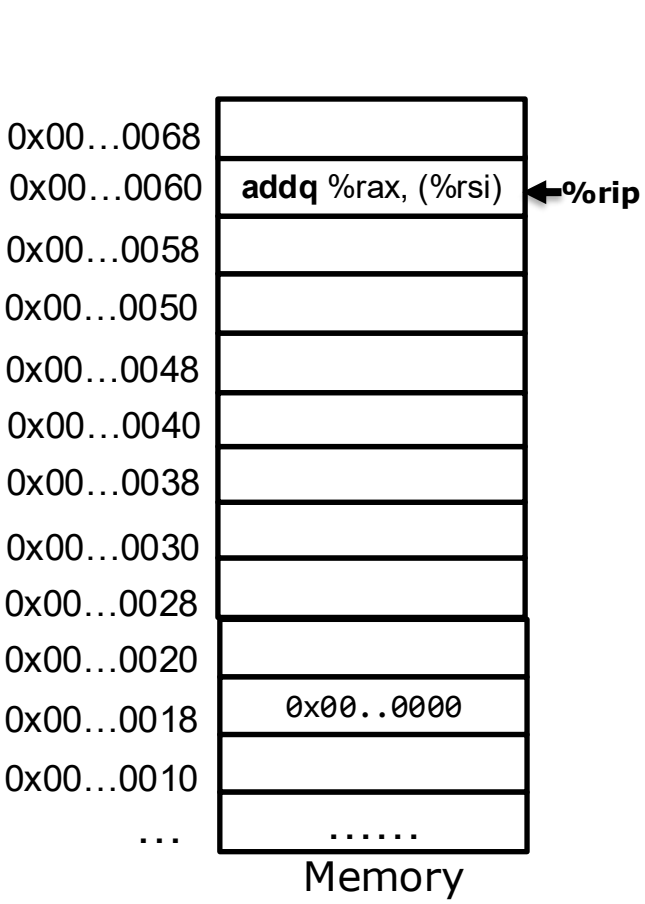
flag	status
ZF (Zero Flag)	set if the result is zero.
SF (Sign Flag)	set if the result is negative.
CF (Carry Flag)	Overflow for unsigned-integer arithmetic
OF (Overflow Flag)	Overflow for signed-integer arithmetic

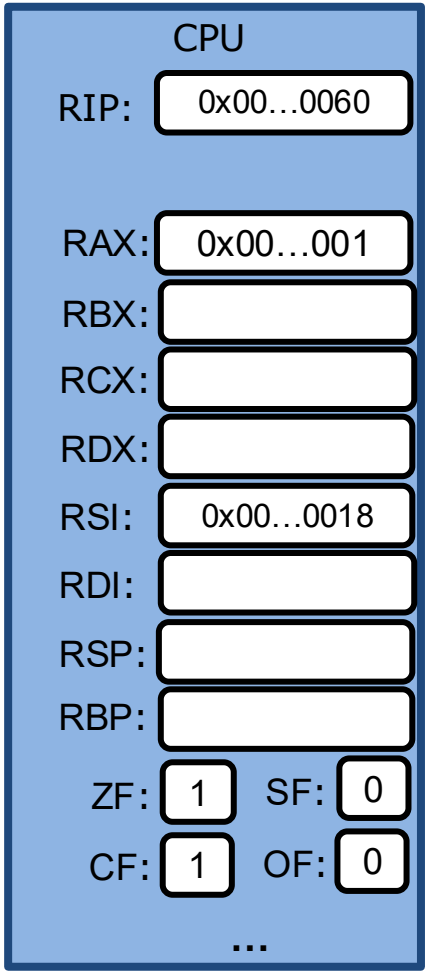
Set by arithmetic instructions, e.g. add, inc, and, sal

Not set by **`lea, mov`**


0x00…0068
0x00…0060
addq %rax, (%rsi)
%rip
0x00…0058
0x00…0050
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0xff…ffffff
0x00…0010
…
……
Memory


CPU
RIP:
0x00…0060
RAX:
0x00…001
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
RSP:
RBP:
ZF:
0
SF:
0
CF:
0
OF:
0
…

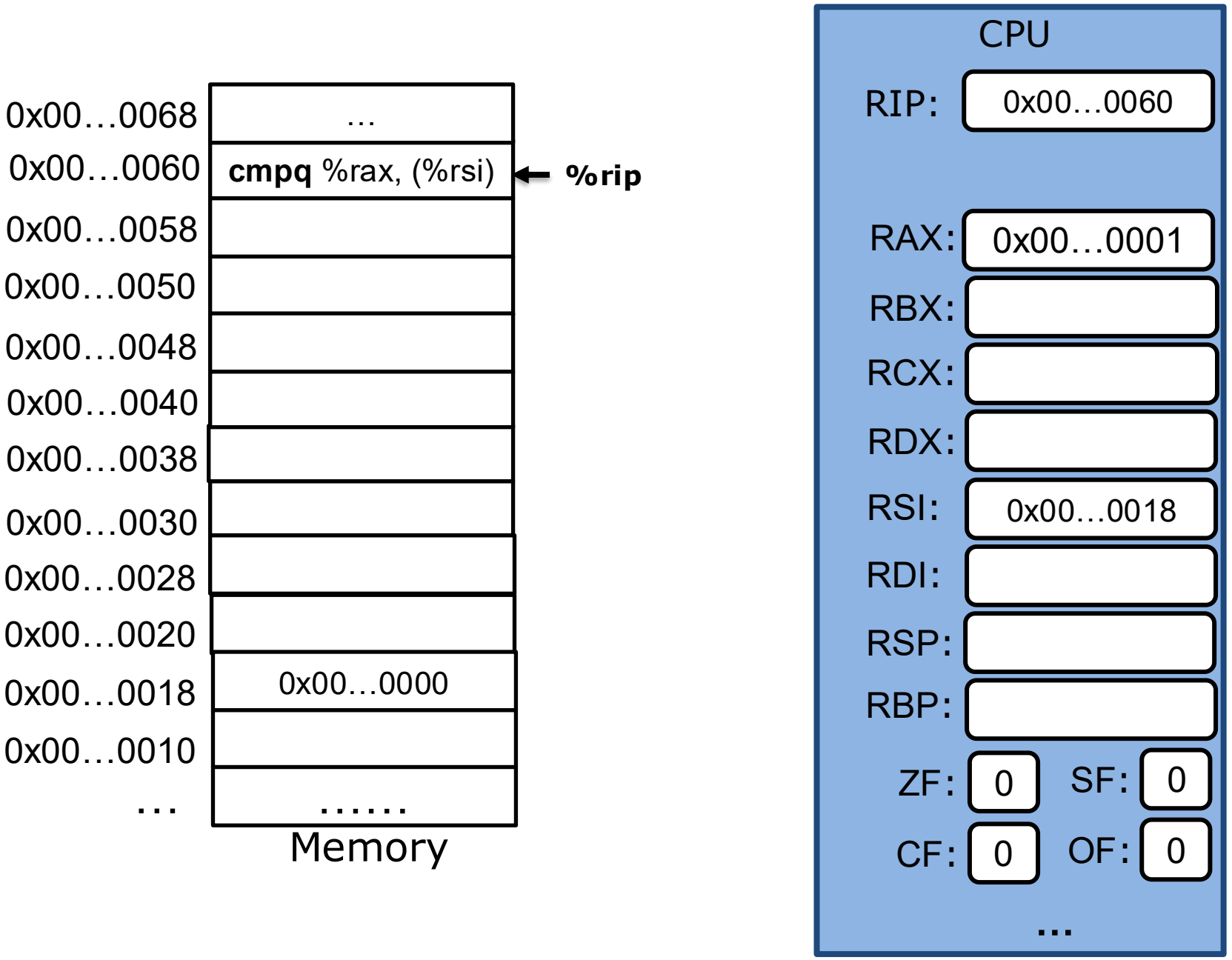

0x00…0068
0x00…0060
addq %rax, (%rsi)
%rip
0x00…0058
0x00…0050
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00..0000
0x00…0010
…
……
Memory

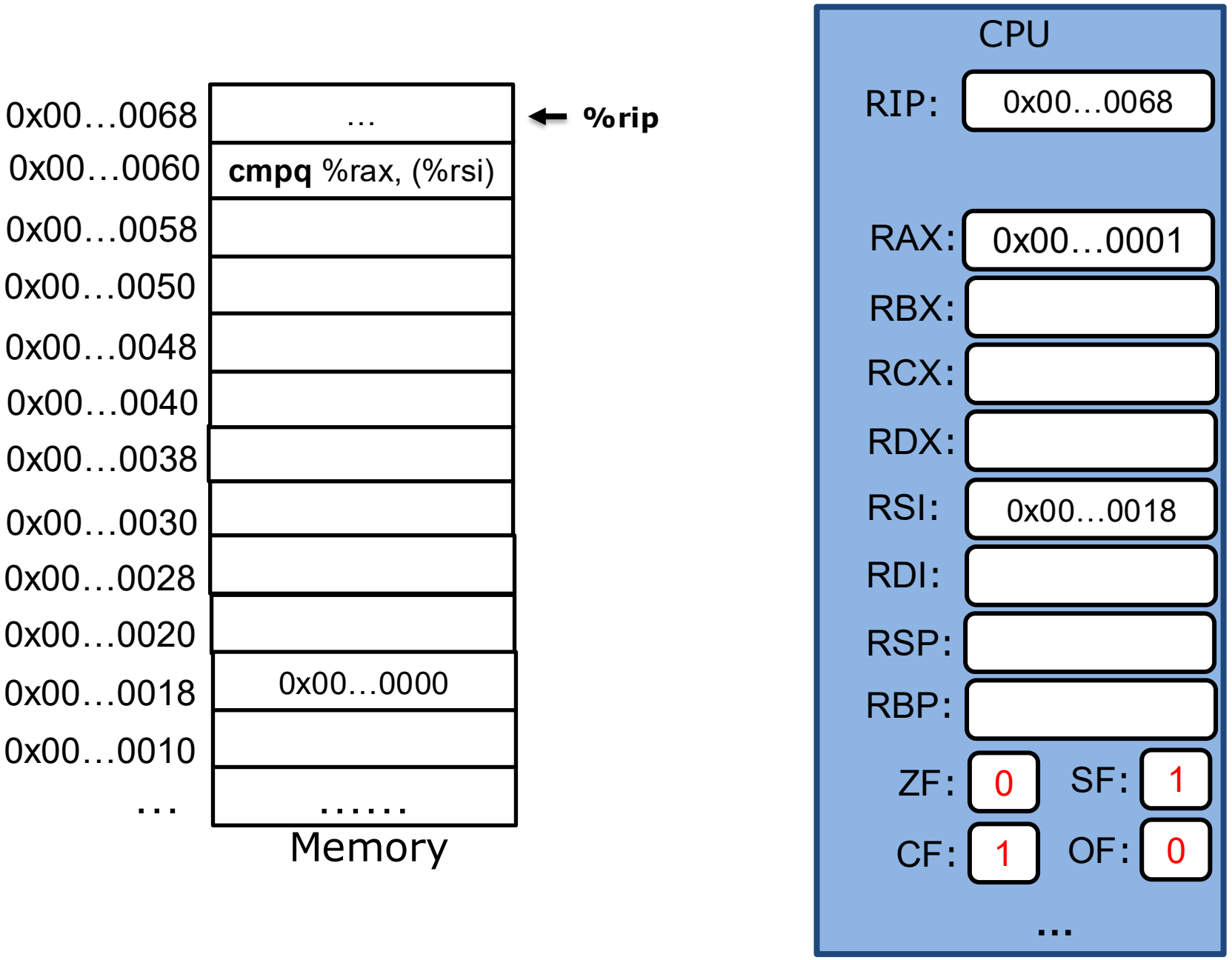

CPU
RIP:
0x00…0060
RAX:
0x00…001
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
RSP:
RBP:
ZF:
1
SF:
0
CF:
1
OF:
0
…

Special instructions that set RFLAGS: cmp

`cmpq` `src, dst`

- Set CF, ZF, SF and OF like **`subq`** `src, dst` except `dst` is unchanged


0x00…0068
…
0x00…0060
cmpq %rax, (%rsi)
%rip
0x00…0058
0x00…0050
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0000
0x00…0010
…
……
Memory
CPU
RIP:
0x00…0060
RAX:
0x00…0001
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
RSP:
RBP:
ZF:
0
SF:
0
CF:
0
OF:
0
…


0x00…0068
…
%rip
0x00…0060
cmpq %rax, (%rsi)
0x00…0058
0x00…0050
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0000
0x00…0010
…
……
Memory
CPU
RIP:
0x00…0068
RAX:
0x00…0001
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
RSP:
RBP:
ZF:
0
SF:
1
CF:
1
OF:
0
…

Special instructions that set RFLAGS: test

`testq` `src, dst`

- Set ZF, SF like **`andq`** `src, dst` except `dst` is unchanged

Questions

`testq` `%rax, %rax`

- When is ZF set?
- When is SF set?

Questions

testq %rax, %rax

- When is ZF set? val(%rax) = 0x0
- When is SF set? val(%rax) < 0

Instructions that read RFLAGS: set

setX `dst`

- Set `dst (%al,or %bl,..etc)` to 1 or 0 based on condition given by suffix X
  - Truthfulness of condition depends on status flags in RFLAGS.
- dst is a 1-byte register (e.g. %al).
  - The upper 56 bits of the 64-bit dst register remain unchanged.

setX dst

```
cmpq a, b
setX dst
```

0x1

0x80000000

SF:true, **OF**:false
SF:false, **OF**:true

```
cmpq %eax, %ebx
```

setX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF)&~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF)|ZF	Less or Equal (Signed)
seta	~CF&~ZF	Above (unsigned)
setb	CF	Below (unsigned)

b < a

Example

```
long gt (long x, long y)
{
  return x > y;
}
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value



Least significant byte of %rax register

```
cmpq   %rsi, %rdi    # cmpq y, x
setg   %al           # set when x > y
movzbq %al, %rax     # zero extend %rax
```

Today's lesson plan

How is non-linear control flow realized?

- RFLAGS register and how arithmetic instructions set it
- Special instructions that also set RFLAGS
  - `cmp, test`
- Instructions that read RFLAGS to set register values
  - `set`
- Instructions that (read RFLAGs to) set %rip
  - `jmp`

(Unconditional) jump instruction

jmp label

- Change %rip to the address specified by label
- jmp is like ***goto***

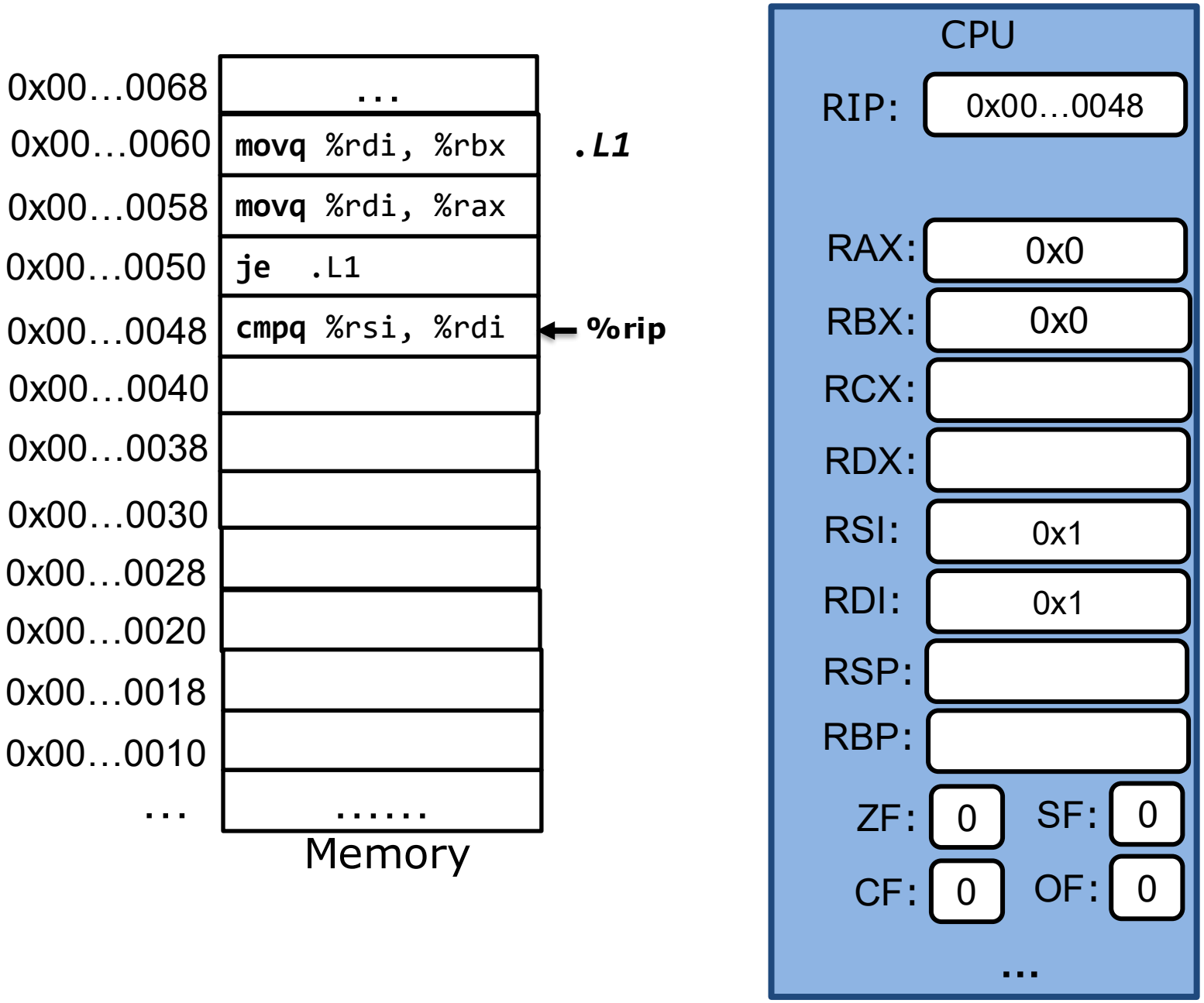
```
  addq %rsi, %rdi
  jmp  .L1
  movq %rdi, %eax
.L1
  movq %rdi, %rbx
```

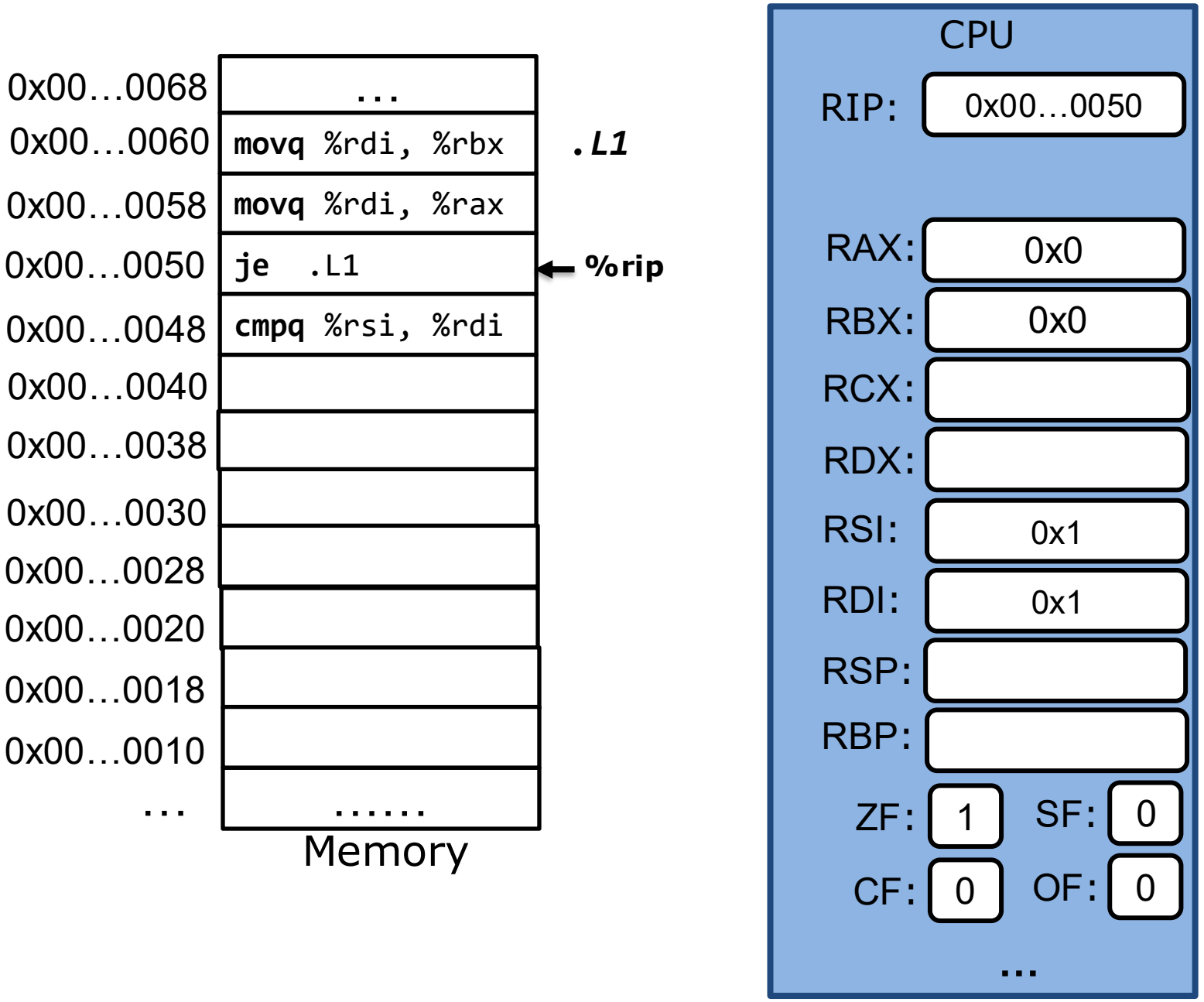
Conditional jump instruction

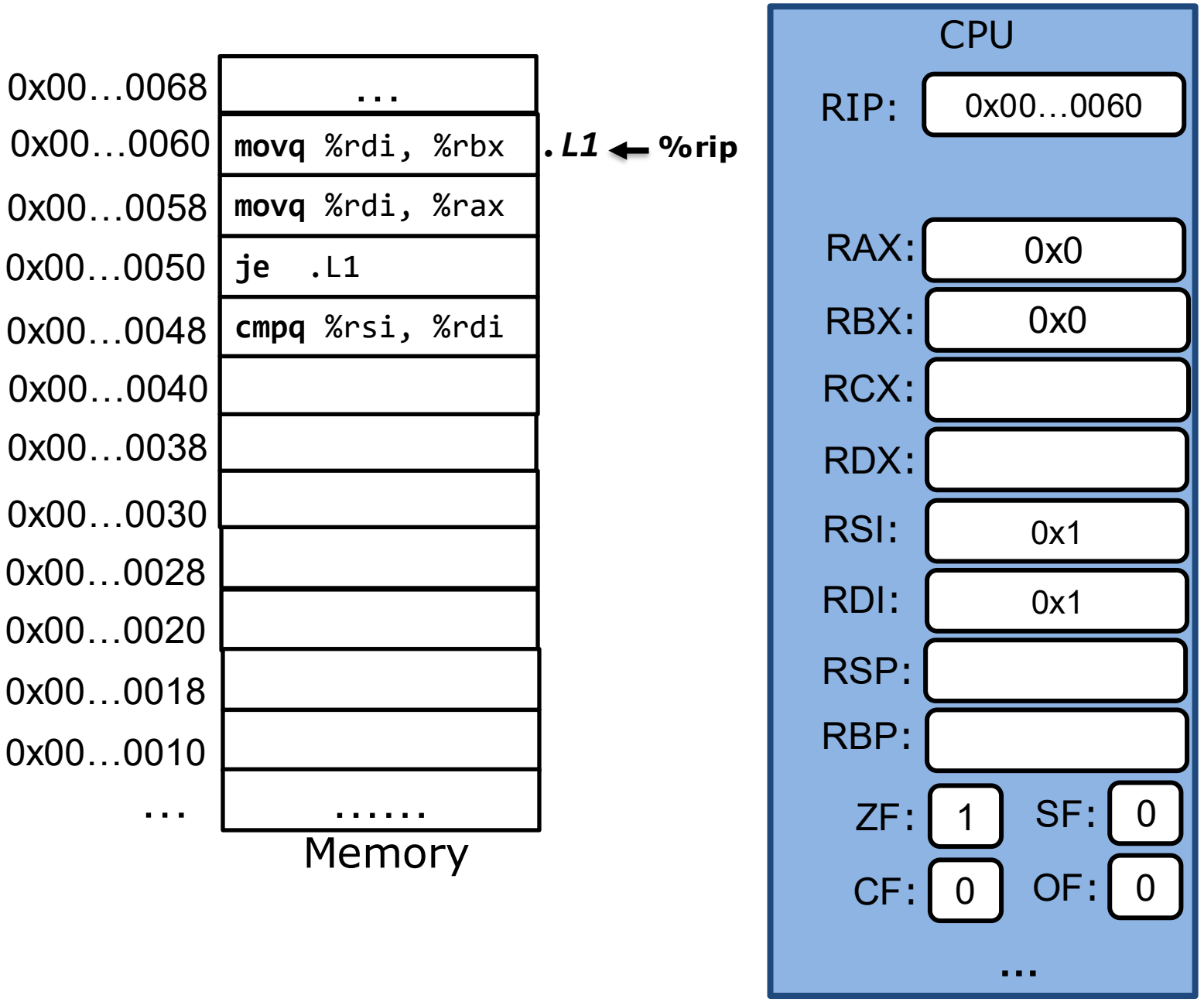
`jX label`

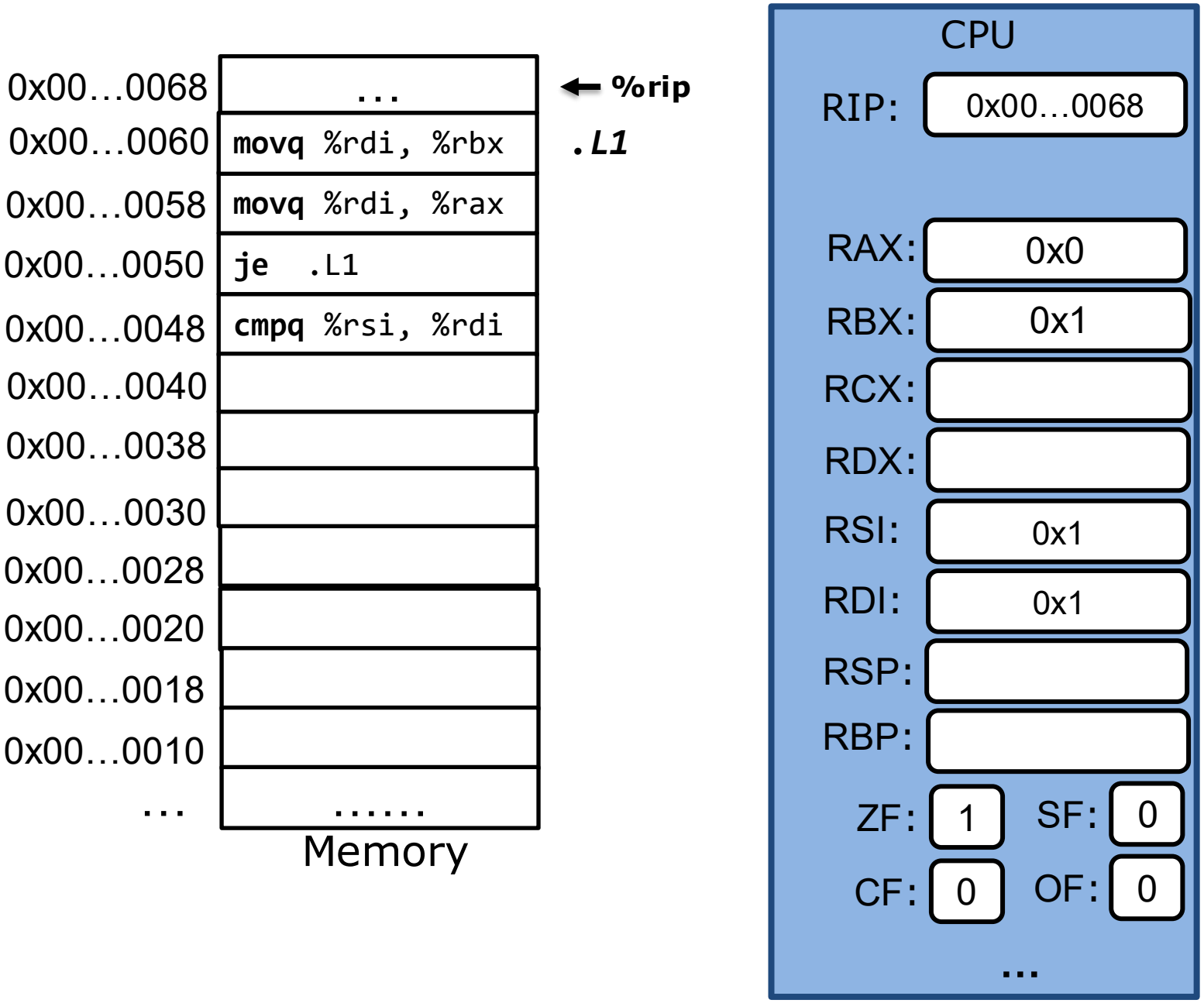
- If condition **X** is met, jump to the label

jX	Condition	Description
`je`	`ZF`	Equal / Zero
`jne`	`~ZF`	Not Equal / Not Zero
`js`	`SF`	Negative
`jns`	`~SF`	Nonnegative
`jg`	`~(SF^OF)&~ZF`	Greater (Signed)
`jge`	`~(SF^OF)`	Greater or Equal (Signed)
`jl`	`(SF^OF)`	Less (Signed)
`jle`	`(SF^OF)|ZF`	Less or Equal (Signed)
`ja`	`~CF&~ZF`	Above (unsigned)
`jb`	`CF`	Below (unsigned)


0x00…0068
...
0x00…0060
movq %rdi, %rbx
.L1
0x00…0058
movq %rdi, %rax
0x00…0050
je .L1
0x00…0048
cmpq %rsi, %rdi
%rip
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
...
.....
Memory
CPU
RIP:
0x00…0048
RAX:
0x0
RBX:
0x0
RCX:
RDX:
RSI:
0x1
RDI:
0x1
RSP:
RBP:
ZF:
0
SF:
0
CF:
0
OF:
0
...


0x00…0068
...
0x00…0060
movq %rdi, %rbx
.L1
0x00…0058
movq %rdi, %rax
0x00…0050
je .L1
%rip
0x00…0048
cmpq %rsi, %rdi
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
...
.....
Memory
CPU
RIP:
0x00…0050
RAX:
0x0
RBX:
0x0
RCX:
RDX:
RSI:
0x1
RDI:
0x1
RSP:
RBP:
ZF:
1
SF:
0
CF:
0
OF:
0
...


0x00…0068
...
0x00…0060
movq %rdi, %rbx
.L1
%rip
0x00…0058
movq %rdi, %rax
0x00…0050
je .L1
0x00…0048
cmpq %rsi, %rdi
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
...
.....
Memory
CPU
RIP:
0x00…0060
RAX:
0x0
RBX:
0x0
RCX:
RDX:
RSI:
0x1
RDI:
0x1
RSP:
RBP:
ZF:
1
SF:
0
CF:
0
OF:
0
...


0x00…0068
...
%rip
0x00…0060
movq %rdi, %rbx
.L1
0x00…0058
movq %rdi, %rax
0x00…0050
je .L1
0x00…0048
cmpq %rsi, %rdi
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
...
.....
Memory
CPU
RIP:
0x00…0068
RAX:
0x0
RBX:
0x1
RCX:
RDX:
RSI:
0x1
RDI:
0x1
RSP:
RBP:
ZF:
1
SF:
0
CF:
0
OF:
0
...

How “if..else..” statement works

```
long compare(long x, long y)
{
  long result;
  if (x > y)
    result = 1;
  else
    result = 0;
  return result;
}
```

gcc –Og –S compared.c

```
compare:
    cmpq    %rdi, %rsi
    jge     .L3
    movl    $1, %rax
    ret
.L3:
    movl    $0, %rax
    ret
```

Register	Use(s)
%rdi	Argument **x**
%rsi	Argument **y**
%rax	Return value

How “while” works

logical right shift

```
long bit_length(unsigned long x)
{
  long s = 0;
  while (x != 0) {
    x = x >> 1;
    s++;
   }
   return s;
}
```

gcc –Og –S count.c

```
bit_length:
    movq $0, %rax
    jmp .L2
.L3:
    shrq %rdi
    addq $1, %rax
.L2:
    testq %rdi, %rdi
    jne .L3
    ret
```

Sets RFLAGS based on result of logical AND

Register	Use(s)
%rdi	Argument x
%rax	Return value

“While” Translation example

```
long count(unsigned long x)
{
  long cnt = 0;
  while (x != 0) {
    x = x >> 1;
    cnt++;
   }
   return cnt;
}
```

```
count:
    movq $0, %rax
    jmp .L2
.L3:
    shrq %rdi
    addq $1, %rax
.L2:
    testq %rdi, %rdi
    jne .L3
    ret
```

```
long cnt = 0;
goto .L2
.L3:
  x = x >> 1
  cnt = cnt + 1
.L2:
  if x != 0
    goto .L3

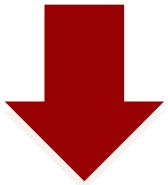
return cnt
```

Register	Use(s)
%rdi	Argument **x**
%rax	Return value

"For" Loop translation

For Version

```
for (Init; Test; Update)
    Body
```



While Version

```
Init;
while (Test) {
    Body
    Update;
}
```

“Loop” Translation example

- `gcc –Og –S *.c`

```
long sum(long n)
{
  long s = 0;
  for (long i=0; i<n; i++){
    s += i;
  }
  return s;
}
```

```
sum:
    movq $0, %rdx
    movq $0, %rax
    jmp .L5
.L6:
    addq %rdx, %rax
    addq $1, %rdx
.L5:
    cmpq %rdi, %rdx
    jl  .L6
    ret
```

Register	Use(s)
%rdi	n
%rax	Return value (s)

Summary

- How CPU implements non-linear control flow
- Special register RFLAGS encodes status flags (ZF, SF, CF, OF)
- `cmp` (or `test`) followed by `jmp` (or `set`)